

Kuromaku

A versioned marketing memory with provenance, staleness propagation, and a planner that reports what it cannot do.

Built as a rebuild of Okara, an "AI CMO" product, with the architectural changes described in section 2. The seed workspace is ShogunAI, a pre-launch macOS app that captures work context into a private local memory.

Live instance: <https://kuromaku-nine.vercel.app>

Repository: <https://github.com/gsaianimesh/kuromaku>

Engineering documentation: docs/ in the repository, sixteen files

Demo walkthrough: section 4 of this document

o. How to read this

This document is the demo. There is no video, so section 4 carries the full walkthrough: every screen, in the order you would visit it, with enough narration that you never need to open the application to follow what happened.

The other sections give it context. Section 1 says what the system is. Section 2 says what it was built against, which is a list of specific defects observed in a real product. Section 3 explains how it fits together. Section 5 covers verification. Section 6 is the honest list of what is missing.

About the screenshots

Every image in this document was captured from the running application against its real database, by a committed script.

```
scripts/screenshots.ts  nineteen static shots
scripts/pairs.ts        before and after pairs for state changes
```

Both scripts navigate to a real route, wait for a selector proving the expected content actually rendered, and **refuse to capture** if it did not. A missing state produces a failed run and a named error, not a substituted placeholder. The screenshot script also scans the settings page for anything shaped like an API key and aborts rather than writing an image with a credential in it. Nothing here was mocked, staged in a fake page, or edited after capture. Regenerate the set with `npm run dev`, then `npm run screenshots` and `npm run pairs`.

Because a static document cannot show a transition, the five state changes that matter most appear twice, labelled *before* and *after*. In every case the change between the two shots was made by calling the real code path, never by writing the after state directly into the database.

What you can check yourself

At the live URL, without any setup:

- `/memory` shows the compiled records with their sources and confidence. Compare any record to the same one in this document.
- `/planner` shows which prioritised channels have no agent.

- `/api/v1/memory` returns the same records as JSON, each carrying its sources and an `unsourced` flag.
- `/api/mcp` returns the MCP tool manifest.
- `/health` reports environment, database, migration and encryption checks, and answers 503 if any fails.

In the repository:

- `docs/` explains every claim in this document at implementation depth, with file paths.
- `npm run verify` runs the infrastructure checks with no model calls.
- `npm run e2e` walks the whole pipeline against real model calls.
- `npm run eval` runs the golden set.

1. What this is

Kuromaku crawls a company website, compiles the extracted text into a set of versioned records called a memory, and runs channel agents that draft marketing work from that memory. Drafts go to a human. Published work accumulates observations, and those observations change what gets scheduled next.

The insight worth keeping from the product it rebuilds is the shared strategy layer. Agents do not each re-derive context. They read one compiled set of records, which is why output stays consistent across channels. Everything else in this system exists to fix something that layer does not do.

Five commitments

Each is visible in the interface, not only in the code.

Provenance on every fact. No memory record exists without a source URL or an explicit human assertion, plus a confidence value. A record the compiler cannot ground is still written, flagged, and capped below 0.5 confidence.

Versioned memory with staleness propagation. Records are append only. Editing supersedes rather than overwrites. Every artifact records which memory records it came from, and editing a record marks derived artifacts stale, transitively.

The agent set is derived from the strategy. The planner reads compiled channel priorities and compares them against a code-seeded agent registry. A prioritised channel with no agent becomes a visible coverage gap, not silence.

Evidence, not justification. Every draft ships with the memory record ids, source links and data points it used. Each is clickable and resolves to something real.

Performance closes the loop. Observations feed the planner. A channel whose recent drafts have no observations stops being scheduled, and the planner says why.

What it explicitly does not do

These are enforced, not merely unbuilt.

It does not publish anything. No code path makes an authenticated write to any platform. `markAsPosted` records a URL a human supplies, and rejects any artifact that is not already approved.

It does not display an unmeasured number. Where nothing was observed the interface says so in words. A model with no entry in the pricing table produces a null cost that renders as "unpriced", never as \$0.00.

It does not authenticate users. Version one is single tenant with no login. Every table carries a workspace id, so multi tenancy is a routing change rather than a migration, but no such routing exists.

2. What it was built against

The following defects were observed in a first hand session with Okara. They are not from its marketing. Each one became a requirement, and each is answered by a specific mechanism.

Strategy and execution were disconnected. Its own strategy ranked Product Hunt third and founder communities fourth. It had no agent for either, while shipping UGC video and influencer agents its strategy never asked for. The agent catalogue was fixed and unrelated to the plan.

Answered by: the planner compares compiled priorities against the registry and writes a coverage gap where nothing can execute. Section 4, step 3.

The thirty day roadmap was a dead document. Empty checkboxes in a PDF. No state, nothing executing it.

Answered by: roadmap items compile into memory records, and the planner turns them into real jobs with real status.

No staleness tracking. Editing the ICP left every derived draft unchanged and unflagged.

Answered by: records are append only, and editing one marks every artifact derived from it stale, through the derivation chain. Section 4, step 2.

Rationale without evidence. Drafts carried a prose "why this works" panel with no links, no named threads, no data, and no way to tell which memory record informed the choice.

Answered by: structured evidence on every draft, each item resolving to a memory record page or a real URL. Section 4, step 4.

Fabricated metrics. Unpublished drafts rendered invented like and view counts.

Answered by: nothing displays a number that was not observed. Empty states say why they are empty. Section 4, step 5.

Duplicate execution. The entire onboarding pipeline ran twice in one session, including all research searches. One search query fired twice inside a single run.

Answered by: every job carries an idempotency key enforced by a partial unique index, and research is cached and deduplicated by normalised query hash.

No provenance. Facts were asserted flat. It listed integrations and a latency claim with no source. Wrong facts got baked into memory and repeated by every agent afterwards.

Answered by: citations are resolved against what the model was actually shown. A hallucinated source index is dropped and the record is capped below 0.5 confidence. Section 4, step 1.

Voice inferred from a landing page. Socials were skipped and it proceeded, inferring brand voice from marketing copy.

Answered by: voice rules compile per locale. A locale with no source material in that language yields low confidence rules that say the material is missing, rather than translated rules presented as observed.

Performance was optional and unused. The whole pipeline completed with Search Console and Analytics skipped, and nothing degraded.

Answered by: observations are a planner input, and their absence changes scheduling. Section 4, step 5.

3. How it fits together

Next.js on the App Router, TypeScript, Tailwind, Neon Postgres through Drizzle, Zod for validation. Model calls go to Groq, with Anthropic behind the same interface. Jobs run on a Postgres queue rather than a third party service.

The pipeline

```
crawl  →> sources      pages, deduplicated by content hash
      |
compile →> memory_records + record_sources + record_derivations
      |               nine record types, provenance, derivation graph
plan   →> jobs         with a plain language reason
      |               or coverage_gaps where no agent exists
run    →> artifacts + artifact_evidence
      |               drafted, critiqued, evidence attached
review →> reviews      approve, edit with distance, or reject
      |
publish →> external_url a human posts it and confirms
      |
observe →> observations measured figures, never generated
      |
      └─> back into plan
```

Where the interesting decisions are

Claiming a job is one statement, not a transaction. The Neon HTTP driver runs each statement in its own implicit transaction, so a standalone `SELECT ... FOR UPDATE SKIP LOCKED` would drop its lock before the follow up update and two workers could claim the same row. The select is nested inside the update:

```
UPDATE jobs
SET status = 'running', locked_at = now(), attempts = attempts + 1
WHERE id = (
  SELECT id FROM jobs
  WHERE status = 'queued' AND run_after <= now()
  ORDER BY run_after ASC, created_at ASC
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
RETURNING *;
```

The idempotency index is partial. It applies only while a job is queued or running:

```
CREATE UNIQUE INDEX "jobs_idempotency_key_uq"
ON "jobs" USING btree ("idempotency_key")
WHERE status in ('queued', 'running');
```

The queue's job is preventing concurrent duplicate execution. Terminal rows release their key because re-running is required behaviour: re-compiling must supersede rather than duplicate, and re-crawling must be possible. Not scheduling redundant work is a planning decision and lives in the planner, which has the history to judge it.

Provenance is enforced after the model returns, never trusted from it. Each stage gives the model a numbered source list. The model returns indices. Those indices are then resolved against the list it was actually shown:

```
for (const idx of record.sourceIndices) {  
  const source = sourceRows[idx];  
  if (source) {  
    citations.push({ sourceId: source.id, url: source.url, snippet: record.snippet });  
  }  
}
```

An index out of range resolves to `undefined` and is dropped. A URL the model invented is not found in the search results and is dropped. What is not dropped is the record itself.

Only three of nine compile stages receive raw page text. Product facts, ICP segments and voice rules read the company's own prose. The rest reason from records already compiled, which is the shared strategy layer doing its job, and also what keeps each prompt inside the provider's per minute token budget.

4. Guided walkthrough

Five steps, one per commitment, in the order you would actually visit them.

Step 1. Provenance on every fact

Start at `/memory`. This is the compiled memory: nine record types, each row carrying a value, a confidence score, a locale, an origin and a version.

kuzomaku v0
Overview
Sources
Memory
Review
Planner
Publish
Metrics
Jobs
Settings
Health

46 active

17 unsourced

avg confidence 0.69

en

ja

COMPILER 8 source(s) available

Queue compile

Run compile now

Re-compiling supersedes records rather than duplicating them.

PRODUCT FACTS 8 records

contextual-reply-drafting 0.90 en compiled v6 history

```
{
  "fact": "ShogunAI drafts replies based on your saved context",
  "category": "capability"
}
```

source: [ShogunAI — Memory that acts](#)
"Reply drafted"

source: [ShogunAI — Private AI Memory for Work Context · ShogunAI](#)
"Reply drafted"

source: [ShogunAI — 仕事の文脈を記憶するプライベートAI · ShogunAI](#)
"Reply drafted"

edit

hybrid-search 0.95 en human v7 history

```
{
  "fact": "ShogunAI features hybrid search across everything you touched",
  "note": "Corrected during review.",
  "category": "capability",
  "correctedByHuman": true
}
```

source: Asserted by a human via the memory editor.

edit

macos 0.90 en compiled v6 history

```
{
  "fact": "ShogunAI is available for macOS",
  "category": "platform"
}
```

source: [ShogunAI — Memory that acts](#)
"macOS"

source: [ShogunAI — Private AI Memory for Work Context · ShogunAI](#)
"macOS"

source: [ShogunAI — 仕事の文脈を記憶するプライベートAI · ShogunAI](#)
"macOS"

edit

Note the header counts: active records, how many are unsourced, average confidence, and the locales present. Each type is its own panel with its own unsourced count in the panel hint, so the quality of the memory is legible before reading a single record.

Open any product fact and look underneath the value.

contextual-reply-drafting
0.90
en
compiled
v6
history

```
{
  "fact": "ShogunAI drafts replies based on your saved context",
  "category": "capability"
}
```

source: [ShogunAI — Memory that acts](#)
"Reply drafted"

source: [ShogunAI — Private AI Memory for Work Context · ShogunAI](#)
"Reply drafted"

source: [ShogunAI — 仕事の文脈を記憶するプライベートAI · ShogunAI](#)
"Reply drafted"

edit

Note the badge row: confidence 0.90, locale en, origin compiled, version v6. Below the JSON value are the resolved sources, each a clickable link to the page it came from, and beneath each one, in quotes, the snippet the model cited as supporting text. That snippet is text the model claimed appears in that source.

Now find a row badged unsourced. There will be several.

```
designers 0.40 en compiled v4 unsourced history
{
  "segment": "designers",
  "painPoints": [
    "Difficulty organizing design files and assets",
    "Challenges in collaborating with team members",
    "Inefficient use of time due to manual note-taking"
  ],
  "description": "People responsible for creating visual and interactive designs",
  "whereTheyGather": [
    "Design communities",
    "Behance"
  ]
}
```

No source. This record is not grounded in any crawled page or search result — treat it as an unverified inference.

edit

Note the confidence reads 0.40. That is not a number the model chose. It is a cap applied at write time to any record with no resolvable citation:

```
// Unsourced records are capped below 0.5 regardless of what the model claimed.
const confidence =
  citations.length === 0 ? Math.min(emitted.confidence, 0.4) : emitted.confidence;
```

The record was not dropped. Dropping it would lose information and make the memory look cleaner than it is. It was written, flagged and capped, and the warning tells a reader what they are looking at: an unverified inference, not a fact.

The competitors section is worth a look for the opposite reason.

```
joplin 0.80 en compiled history
{
  "url": "https://joplinapp.org/",
  "name": "joplin",
  "overlap": "note taking",
  "positioning": "open-source note taking app",
  "differenceFromUs": "local storage, end-to-end encryption"
}
source: https://zapier.com/blog/best-note-taking-apps
  "Joplin"
source: https://www.pcmag.com/picks/the-best-note-taking-apps
  "Joplin"
edit
```

Note this record carries a real source URL from a web search, not from the company's own site. Competitors are the one stage that runs research. When no search provider is configured, this section produces nothing at all rather than a list of plausible sounding names, and the compile log records why.

The defect this answers. Okara asserted facts flat, with no source. It listed integrations and a latency claim that were simply wrong, and every agent then repeated them. Here an ungrounded claim cannot hide: it is capped by the writer rather than the model, it is labelled in the interface, and the count of unsourced records is on the page header.

Step 2. Versioned memory, and what an edit invalidates

This is the step that matters most. Records are append only: nothing is updated in place except the status column of the row being retired, and editing produces a new version that supersedes the old one. Open a record's history at `/memory/<id>`.

kuzomaku v0

OverviewSourcesMemoryReviewPlannerPublishMetricsJobsSettingsHealth

← memory hybrid-searchProduct factsen

VERSION HISTORYappend only — editing supersedes, it never overwrites

v7 · activehumanconfidence 0.952026-08-13T19:11:21.900Z

```
{
  "fact": "ShogunAI features hybrid search across everything you touched",
  "note": "Corrected during review.",
  "category": "capability",
  "correctedByHuman": true
}
```

Asserted by a human via the memory editor.

v6 · supersededcompiledconfidence 0.902026-08-13T18:05:39.134Z

```
{
  "fact": "ShogunAI features hybrid search across everything you touched",
  "category": "capability"
}
```

ShogunAI — Memory that acts
ShogunAI — Private AI Memory for Work Context · ShogunAI
ShogunAI — 仕事の文脈を記憶するプライベートAI · ShogunAI

v5 · supersededcompiledconfidence 0.902026-08-13T18:03:25.003Z

```
{
  "fact": "ShogunAI connects tools and builds a live memory to do work",
  "category": "capability"
}
```

ShogunAI — Memory that acts
ShogunAI — Private AI Memory for Work Context · ShogunAI
ShogunAI — 仕事の文脈を記憶するプライベートAI · ShogunAI

v4 · supersededcompiledconfidence 0.502026-08-13T17:48:36.956Z

```
{
  "fact": "Provides hybrid search across all captured content",
  "category": "capability"
}
```

ShogunAI — Memory that acts

v3 · supersededcompiledconfidence 0.902026-08-13T14:17:10.586Z

```
{
  "fact": "ShogunAI performs hybrid search across everything you touched.",
  "category": "capability"
}
```

ShogunAI — Memory that acts

v2 · supersededcompiledconfidence 0.952026-08-13T13:46:46.870Z

```
{
  "fact": "ShogunAI performs hybrid search across all tools and documents you have touched.",
  "category": "capability"
}
```

ShogunAI — Memory that acts

v1 · supersededcompiledconfidence 0.902026-08-13T13:40:58.276Z

```
{
  "fact": "ShogunAI provides hybrid search across everything the user has touched.",
  "category": "capability"
}
```

ShogunAI — Memory that acts

N

Note that the page shows every version, newest first, and that nothing was deleted: v7 is active with origin `human` , and v6 down to v1 are all superseded, each keeping its own sources. Further down the same page, a panel headed "What an edit here would invalidate" lists the records compiled from this one, grouped by how many hops away they are.

Here are the two ends of one chain, side by side.

VERSION HISTORY: THE SAME RECORD BEFORE AND AFTER A HUMAN CORRECTION

BEFORE

```
v6 · superseded  compiled  confidence 0.90  2026-08-13T18:05:39.134Z
{
  "fact": "ShogunAI features hybrid search across everything you touched",
  "category": "capability"
}
ShogunAI — Memory that acts
ShogunAI — Private AI Memory for Work Context · ShogunAI
ShogunAI — 仕事の文脈を記憶するプライベートAI · ShogunAI
```

Note the badge reads *superseded* with origin *compiled*. This is what the compiler produced. It is retained in full, with its own sources, and remains readable forever.

AFTER

```
v7 · active  human  confidence 0.95  2026-08-13T19:11:21.900Z
{
  "fact": "ShogunAI features hybrid search across everything you touched",
  "note": "Corrected during review.",
  "category": "capability",
  "correctedByHuman": true
}
Asserted by a human via the memory editor.
```

Note the badge reads *active* with origin *human*, and the version number has advanced. The value differs from the version above it. A human corrected the record, and that correction became a new row rather than an overwrite.

The human version still carries provenance. `editRecord` writes a source row recording the assertion:

```
await db.insert(recordSources).values({
  recordId: created.id,
  sourceId: null,
  url: null,
  snippet: "Asserted by a human via the memory editor.",
});
```

Without that row a human edited record would count as unsourced and get a warning, which would be wrong. A person asserting something is a form of provenance, just not a URL.

What the edit does downstream

Here is a draft in the review queue immediately before a memory edit, and the same draft immediately after.

A DRAFT BEFORE AND AFTER A RECORD IT DEPENDS ON WAS CORRECTED

BEFORE

POST · HACKER_NEWS
Launch and community agent · en
critic 0.85
draft

Designers often struggle with organizing their files and assets, which can lead to wasted time and decreased productivity. A hybrid search function that can look across all files, notes, and tools can help alleviate this issue. By having a private and local storage system, designers can also ensure that their data is secure and easily accessible.

EVIDENCE

hybrid-search · product_fact: hybrid-search

designers · icp_segment: designers

private-local-memory-vs-cloud-notes · positioning: private-local-memory-vs-cloud-notes

CRITIC

[medium] use-direct-second-person: The draft uses third-person phrasing (e.g., "Designers often struggle...", "A hybrid search function...") instead of addressing the reader directly with second-person language.

PERFORMANCE

No observations recorded. Nothing is shown here that was not measured.

Approve
Edit and approve
Reject

job 8b107409 · created 2026-08-13T19:10:45.317Z

Note the evidence panel lists memory record keys as links, and nothing is flagged. This draft is ordinary pending work.

AFTER

Stale. The memory this draft rests on has changed since it was written:

- product_fact: hybrid-search was superseded
- icp_segment: designers was compiled from product_fact: hybrid-search, which was superseded (1 hop upstream)
- positioning: private-local-memory-vs-cloud-notes was compiled from product_fact: hybrid-search, which was superseded (1 hop upstream)

Regenerate from current memory

Note the banner names the specific record that changed, not just "this is out of date". Where the change was inherited rather than direct, it says how far upstream it happened and which record on the path was affected. A "Regenerate from current memory" button enqueues a fresh run of the same agent.

Nothing polled and nothing was scheduled. Staleness is computed at edit time by walking a derivation graph the compiler wrote:

```
with recursive downstream(id, depth, path) as (
  select $1::uuid, 0, array[$1::uuid]
  union all
  select rd.derived_record_id, d.depth + 1, d.path || rd.derived_record_id
  from record_derivations rd
  join downstream d on rd.source_record_id = d.id
  where d.depth < 16
  and not rd.derived_record_id = any(d.path)
)
select distinct id from downstream
```

That graph is real: on the current workspace it holds 199 edges, and a single product fact reaches 19 records across three hops, through ICP segments, then positioning, then messaging pillars. Editing the fact marks every artifact citing any of them.

Propagating one hop would have been much easier and would have looked identical in a demo where the draft happens to cite the edited record directly. It would also have reproduced the original defect one level down: correct a product fact, and the positioning built on it keeps looking current.

kuzomaku v0

Overview

Sources

Memory

Review

Planner

Publish

Metrics

Jobs

Settings

Health

Review queue

draft 1

approved 2

published 1

stale 3

RUN WORK

Run queued work

Drains agent and planner jobs so you can watch drafts appear.

POST: HACKER_NEWS Launch and community agent · en

critic 0.85

stale

Stale. The memory this draft rests on has changed since it was written:

- product_fact: hybrid-search was superseded
- icp_segment: designers was compiled from product_fact: hybrid-search, which was superseded (1 hop upstream)
- positioning: private-local-memory-vs-cloud-notes was compiled from product_fact: hybrid-search, which was superseded (1 hop upstream)

Regenerate from current memory

Designers often struggle with organizing their files and assets, which can lead to wasted time and decreased productivity. A hybrid search function that can look across all files, notes, and tools can help alleviate this issue. By having a private and local storage system, designers can also ensure that their data is secure and easily accessible.

EVIDENCE

hybrid-search - product_fact: hybrid-search (superseded)

designers - icp_segment: designers

private-local-memory-vs-cloud-notes - positioning: private-local-memory-vs-cloud-notes

CRITIC

[medium] use-direct-second-person: The draft uses third-person phrasing (e.g., "Designers often struggle..."; "A hybrid search function...") instead of addressing the reader directly with second-person language.

PERFORMANCE

No observations recorded. Nothing is shown here that was not measured.

Approve

Edit and approve

Reject

job — · created 2026-08-13T19:10:45.317Z

COMMENT_ANGLE: HACKER_NEWS Launch and community agent · en

critic 0.90

draft

When running design reviews, it's crucial to keep track of feedback and discussions across different tools. Your AI can help with this by capturing context passively and performing hybrid search across all tools and documents you've touched, making it easier to recall important information.

EVIDENCE

hybrid-search - product_fact: hybrid-search (superseded)

CRITIC

[low] avoid-exaggerated-adjectives: Uses the adjective "crucial" which is an

Note the status counts at the top of the queue have changed. Stale is a first class artifact status, not a warning banner bolted on, which is what lets the queue be filtered and counted by it.

The defect this answers. Editing the ICP in Okara left every derived draft unchanged and unflagged. There was no way to tell which work rested on a fact you had just corrected. Here the edit names its own consequences, and the version you corrected stays readable next to the one that replaced it.

Step 3. The agent set is derived from the strategy

Go to `/planner` . The left of this screen is what the strategy compiled. The right is what can actually execute it.

kuzomaku v0

OverviewSourcesMemoryReviewPlannerPublishMetricsJobsSettingsHealth

Planner

2 open coverage gaps2 registered agents

RUN PLANNER

Run planner nowAlso runs on the 5-minute cron.

COVERAGE GAPS

a prioritised channel with no agent — surfaced, not ignored

reddit

rank 4

open

acknowledge

Ranked 4 in the strategy — Designers and product managers participate in subreddits like r/design, r/productmanagement, and r/startups to discuss workflow tools, offering a relevant outreach point., but no registered agent serves this channel, so nothing can be drafted for it.

linkedin

rank 5

open

acknowledge

Ranked 5 in the strategy — Professional networking on LinkedIn is common among founders and product managers seeking B2B SaaS solutions, making it a viable but higher-effort channel for targeted ads and content., but no registered agent serves this channel, so nothing can be drafted for it.

CHANNEL PRIORITIES VS AGENT COVERAGE

strategy on the left, what can actually execute it on the right

Rank	Channel	Agent	Rationale
1	hacker_news	covered	Founders and product managers frequently browse Hacker News for startup news, tooling, and AI trends, making it a high-fit channel for ShogunAI's core audience.
2	product_hunt	covered	Product managers and early-stage founders monitor Product Hunt launches to discover new productivity tools, providing direct exposure to ShogunAI's value proposition.
3	indie_communities	covered	Indie founder communities (e.g., Indie Hackers) attract entrepreneurs building fast and caring about privacy, aligning with ShogunAI's positioning on private local memory.
4	reddit	no agent	Designers and product managers participate in subreddits like r/design, r/productmanagement, and r/startups to discuss workflow tools, offering a relevant outreach point.
5	linkedin	no agent	Professional networking on LinkedIn is common among founders and product managers seeking B2B SaaS solutions, making it a viable but higher-effort channel for targeted ads and content.
6	seo	covered	Organic search captures users researching "AI note-taking", "private AI memory", and related terms, supporting long-term discovery for all three

Note the coverage column reading covered or no agent . It is computed from the agent registry at request time, so registering an agent changes it immediately, with no migration and no re-compile.

The registry is seeded in code and is deliberately shorter than the channel list the compiler can produce:

```
/**
 * This list is deliberately shorter than the channel list the compiler can
 * produce. That mismatch is the point: a prioritised channel with no agent here
 * becomes a visible coverage gap instead of silently doing nothing.
 */
```

Two agents cover six channels. The compiler can prioritise ten. What happens to the other four is the entire point of this screen.

COVERAGE GAPS

a prioritised channel with no agent — surfaced, not ignored

reddit

rank 4

open

acknowledge

Ranked 4 in the strategy — Designers and product managers participate in subreddits like r/design, r/productmanagement, and r/startups to discuss workflow tools, offering a relevant outreach point., but no registered agent serves this channel, so nothing can be drafted for it.

linkedin

rank 5

open

acknowledge

Ranked 5 in the strategy — Professional networking on LinkedIn is common among founders and product managers seeking B2B SaaS solutions, making it a viable but higher-effort channel for targeted ads and content., but no registered agent serves this channel, so nothing can be drafted for it.

Note each gap carries the rank the strategy assigned and a sentence explaining that nothing can draft for it. These are real: the compiler ranked Reddit fourth and LinkedIn fifth, and the registry contains neither. The planner did not attempt them and did not skip them quietly. It wrote a row that someone has to look at.

In code the gap is the output, and the continue is what makes it so:

```

if (agents.length === 0) {
  await db.insert(coverageGaps).values({ ... })
    .onConflictDoUpdate({ ... });
  result.gaps.push({ channel: entry.channel, rank: entry.rank, rationale });
  continue; // no job is created for this channel
}

```

Where an agent does exist, the job carries the reason it was scheduled.

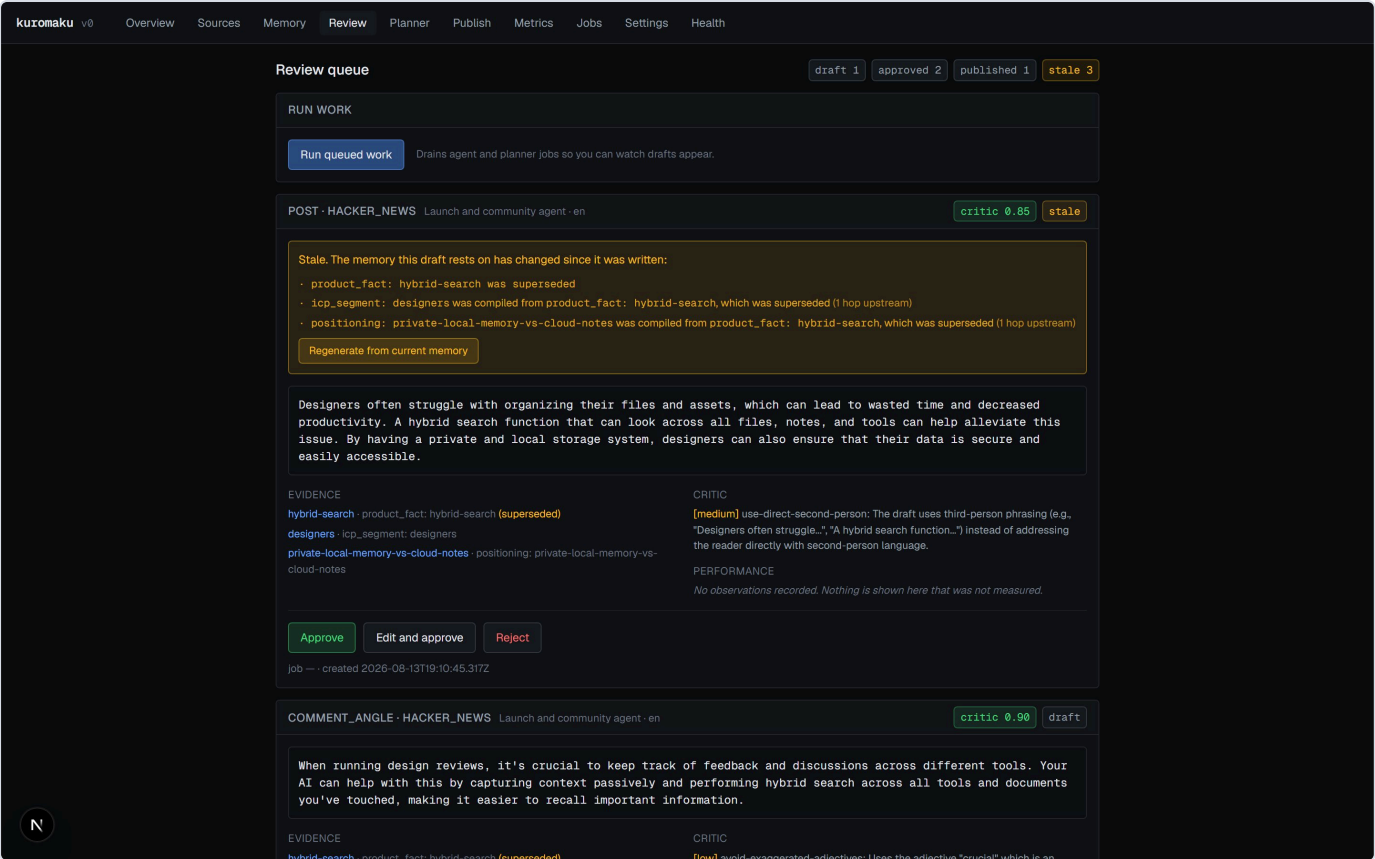
SCHEDULED WORK every job carries the reason it was scheduled		
queued	seo	inspect
Roadmap item "Optimize landing page for privacy-focused SEO keywords": Add content targeting terms like "private AI memory" and "on-device AI assistant," improve meta tags, and publish a supporting blog post referencing the privacy pillar.		
queued	indie_communities	inspect
Roadmap item "Share demo video in Indie Hacker communities": Post a short demo highlighting free early access and the privacy-first design in Indie Hackers and similar indie-founder forums.		
queued	product_hunt	inspect
Roadmap item "Launch ShogunAI on Product Hunt": Submit ShogunAI with screenshots that showcase integrated workflow tools and hybrid search, and encourage the community to up-vote and comment.		
queued	hacker_news	inspect
Roadmap item "Announce ShogunAI early-access on Hacker News": Write a concise post highlighting private-on-device memory and seamless passive capture, and link to the early-access sign-up page.		
queued	seo	inspect
Channel ranked 6 in the compiled strategy (Organic search captures users researching "AI note-taking", "private AI memory", and related terms, supporting long-term discovery for all three segments.). Content agent covers it, and nothing has been drafted for it recently.		
queued	indie_communities	inspect
Channel ranked 3 in the compiled strategy (Indie founder communities (e.g., Indie Hackers) attract entrepreneurs building fast and caring about privacy, aligning with ShogunAI's positioning on private local memory.). Launch and community agent covers it, and 0 of 1 recent artifact(s) here have observations.		
queued	hacker_news	inspect
Channel ranked 1 in the compiled strategy (Founders and product managers frequently browse Hacker News for startup news, tooling, and AI trends, making it a high-fit channel for ShogunAI's core audience.). Launch and community agent covers it, and 1 of 6 recent artifact(s) here have observations.		

Note the reason quotes the compiled rationale verbatim, so the chain from strategy to scheduled work is readable without opening the memory browser. That text is written to the job row at enqueue time. It is stored data, not a sentence regenerated for display.

The defect this answers. Okara's own strategy ranked Product Hunt third and founder communities fourth while shipping agents for neither, and shipping UGC video and influencer agents its strategy never asked for. The catalogue was fixed and unrelated to the plan. Here the plan and the catalogue are compared on one screen, and the difference is recorded rather than hidden.

Step 4. Evidence, not justification

Go to `/review`. This is the daily surface.



Note each artifact shows its channel, kind, agent, critic score and status. Everything needed to triage is on the card.

Beside the content is the evidence panel, and every item in it is a link.

EVIDENCE, AND THE RECORD IT RESOLVES TO

BEFORE

EVIDENCE

hybrid-search · product_fact: hybrid-search (superseded)

designers · icp_segment: designers

private-local-memory-vs-cloud-notes · positioning: private-local-memory-vs-cloud-notes

Note each entry names the record type and key, and the trailing note explains what the item is. An entry attributed by the runner rather than named by the model says so explicitly.

AFTER

The screenshot shows the 'Memory' tab in the 'kuromaku v0' application. The breadcrumb trail is 'memory > hybrid-search > Product facts > en'. The main panel displays a 'VERSION HISTORY' for a fact, with the note 'append only — editing supersedes, it never overwrites'. The history shows five versions (v3 to v7) of a fact. Version v7 is the current active state, marked as 'human' with a confidence of 0.95. It contains a JSON object with fields for fact, note, category, and correctedByHuman. The trail continues from the draft (v7) to the record it used (v6), and so on, showing the lineage of the information.

Version	Status	Confidence	Timestamp
v7	active	0.95	2026-08-13T19:11:21.900Z
v6	superseded	0.90	2026-08-13T18:05:39.134Z
v5	superseded	0.90	2026-08-13T18:03:25.003Z
v4	superseded	0.50	2026-08-13T17:48:36.956Z
v3	superseded	0.90	2026-08-13T14:17:10.586Z

Note that the record you land on carries its own sources, its own confidence, and its own version history. The trail continues: from draft, to the record it used, to the page that record came from.

This is what "evidence, not justification" means in practice. There is no prose panel explaining why the draft is good. There is a list of things it rests on, each of which can be opened, checked and corrected.

The evidence requirement is enforced, not encouraged. The runner throws before persisting an artifact that has none:

```
if (draft.evidence.length === 0) {
  throw new Error(
    `Agent "${agentId}" returned an artifact with no evidence. Every draft must carry the memory records and links it rests on.`
  );
}
```

Beside the evidence sits the critic, which reviewed the draft before any human saw it.

CRITIC

[medium]

 use-direct-second-person: The draft uses third-person phrasing (e.g., "Designers often struggle...", "A hybrid search function...") instead of addressing the reader directly with second-person language.

PERFORMANCE

No observations recorded. Nothing is shown here that was not measured.

Note the violation is specific. It names the voice rule it breaks, quotes the offending phrase, and carries a severity. A draft scoring below 0.7 is revised once automatically and then re-scored, so the number shown always describes the text on screen. When that happens the panel says so.

Cost is not hidden either. Every model call is logged with its tokens and price.

compiler:product_facts

llama-3.3-70b-versatile

compile

18:05:34

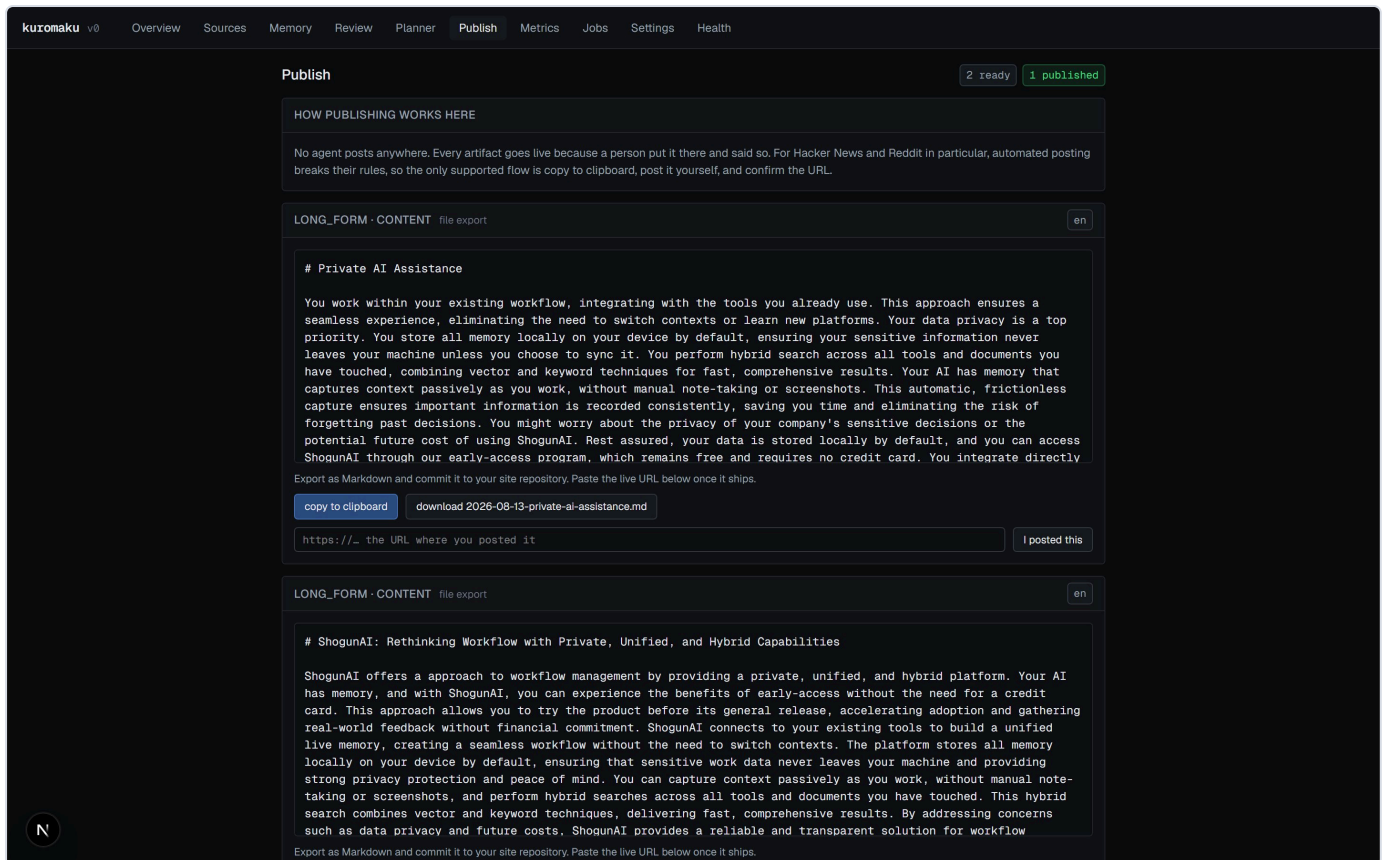
INPUT	OUTPUT	COST	DURATION
2,868	895	\$0.002399	2242ms
▶ prompt (12,041 chars) ▶ raw output (3,345 chars)			

Note the four measured figures, and that the prompt and raw output are both expandable. An unpriced model renders as "unpriced" rather than \$0.00, because a zero would claim the call was free rather than that its model has no price entry.

The defect this answers. Okara's drafts carried a prose "why this works" panel with no links, no named threads, no data, and no reference to which memory record informed the choice. Worse, it rendered invented like and view counts on drafts that had never been published. Here every claim about provenance is a link, and every number is either measured or absent.

Step 5. Performance closes the loop

Go to `/publish` .



Note the panel explaining that no agent posts anywhere. For Hacker News and Reddit the instruction says plainly that automated posting breaks their rules and is not implemented. The only supported flow is copy the text, post it yourself, then paste the resulting URL.

That confirmation is gated in code:

```
if (artifact.status !== "approved") {
  throw new Error(
    `Only an approved artifact can be marked as posted. This one is ${artifact.status}.`,
  );
}
```

A draft cannot skip review, and publishing requires a real URL so that performance can later be attributed to it.

Once something is live, record what actually happened. Here is the performance panel before and after a single observation.

THE PERFORMANCE PANEL WITH NOTHING MEASURED, AND AFTER ONE OBSERVATION

BEFORE

OBSERVATIONS recorded, never generated — nothing here was projected

No performance has been observed yet.

This is deliberately blank rather than showing zeros. A zero would claim something was measured and found to be nothing; an empty state says nothing was measured. Record an observation from the **publish** screen once something is live.

Note it does not show zeros. It says no performance has been observed yet, and explains why that is different: a zero would claim something was measured and found to be nothing, while an empty state says nothing was measured.

AFTER

OBSERVATIONS recorded, never generated — nothing here was projected				
upvotes: 42 across 1				
Metric	Value	Channel	Source	Observed
upvotes	42	hacker_news	manual	2026-08-13 15:22

Note the figure, its metric, its channel and its source. Observations are only ever inserted. No code path derives, estimates or backfills one.

Now the part that makes this a loop rather than a report. Here is the planner scheduling a channel, and the planner declining to schedule that same channel on a later run.

THE PLANNER SCHEDULING A CHANNEL, THEN GATING IT

BEFORE

SCHEDULED WORK every job carries the reason it was scheduled		
queued	seo	inspect
Roadmap item "Optimize landing page for privacy-focused SEO keywords": Add content targeting terms like "private AI memory" and "on-device AI assistant," improve meta tags, and publish a supporting blog post referencing the privacy pillar.		
queued	indie_communities	inspect
Roadmap item "Share demo video in Indie Hacker communities": Post a short demo highlighting free early access and the privacy-first design in Indie Hackers and similar indie-founder forums.		
queued	product_hunt	inspect
Roadmap item "Launch ShogunAI on Product Hunt": Submit ShogunAI with screenshots that showcase integrated workflow tools and hybrid search, and encourage the community to up-vote and comment.		
queued	hacker_news	inspect
Roadmap item "Announce ShogunAI early-access on Hacker News": Write a concise post highlighting private-on-device memory and seamless passive capture, and link to the early-access sign-up page.		
queued	seo	inspect
Channel ranked 6 in the compiled strategy (Organic search captures users researching "AI note-taking", "private AI memory", and related terms, supporting long-term discovery for all three segments.). Content agent covers it, and nothing has been drafted for it recently.		
queued	hacker_news	inspect
Channel ranked 1 in the compiled strategy (Founders and product managers frequently browse Hacker News for startup news, tooling, and AI trends, making it a high-fit channel for ShogunAI's core audience.). Launch and community agent covers it, and 1 of 9 recent artifact(s) here have observations.		

Note the scheduled job and its reason. At this point the channel is ranked, covered by an agent, and nothing blocks it.

AFTER

RUN PLANNER

Planning...

Also runs on the 5-minute cron.

done

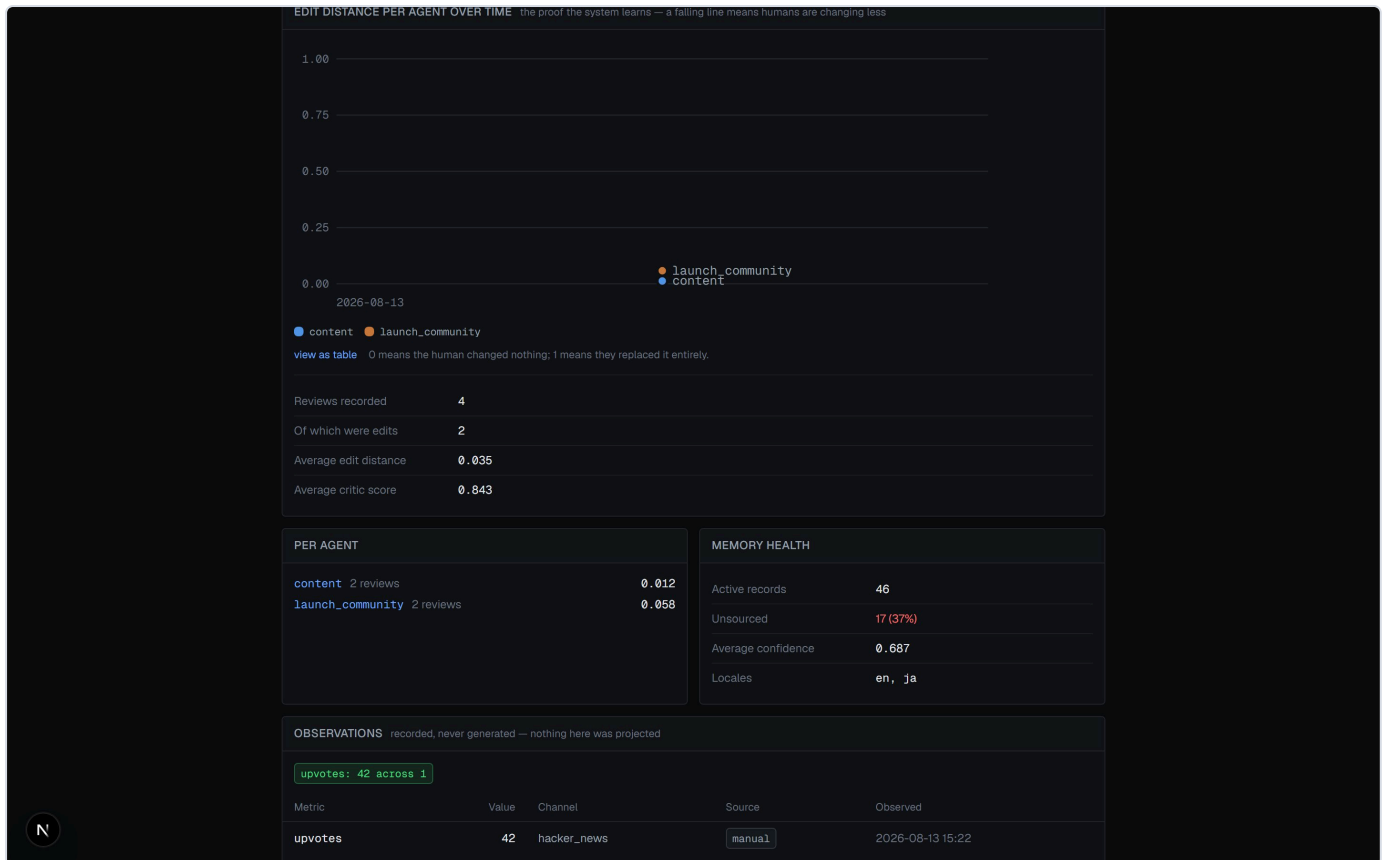
```
skip product_hunt: 3 artifact(s) in the last 14 days and no observations recorded for any of them. Record performance for this channel before drafting more.
skip indie_communities: 3 artifact(s) in the last 14 days and no observations recorded for any of them. Record performance for this channel before drafting more.
coverage gap: reddit (rank 4) has no agent
coverage gap: linkedin (rank 5) has no agent
skip seo: 2 artifact(s) in the last 14 days and no observations recorded for any of them. Record performance for this channel before drafting more.
planner: 0 scheduled, 2 coverage gap(s), 4 skipped
```

Note the reason text: recent artifacts exist in that channel and none of them have any observation recorded, so the planner declines to draft more until someone measures the work already done.

The rule is two conditions, both required:

```
if (recentStats && recentStats.total >= 2 && recentStats.observed === 0) {
  const why =
    `${recentStats.total} artifact(s) in the last ${RECENT_WINDOW_DAYS} days and no observations
    recorded ` +
    `for any of them. Record performance for this channel before drafting more.`;
  result.skipped.push({ channel: entry.channel, why });
  continue;
}
```

A single recent draft is not enough to gate: one draft is not a pattern. And a channel with any observation is not gated regardless of what the number says. The rule is about whether anyone is measuring, not whether the results were good.



Note the memory health panel, and that the edit distance chart draws only the days that actually have reviews. It does not zero fill and does not interpolate, which is why with two reviews on one day it shows two points and no line. The chart is correct. The claim it exists to support needs weeks of data this system has not yet accumulated, and section 6 says so rather than implying otherwise.

The defect this answers. Okara's entire pipeline completed with Search Console and Analytics skipped, and nothing degraded. Performance data was never an input to planning. Here the absence of measurement is itself an input, and it changes what gets scheduled.

5. How it is verified

There is no unit test suite. What exists instead is three executable scripts that assert against a real database and, for two of them, real model calls. That trade and its costs are stated plainly in section 6.

`npm run verify` runs the infrastructure checks with no model calls: environment, connectivity, migration state, the encryption round trip, and the whole queue contract. It reads the raw key column back to confirm the plaintext does not appear in it, and it asserts that two concurrent claims take different rows, a throwing job requeues with backoff, exhausted attempts mark it failed and release its key, and a job whose worker died is recovered rather than stranded. It is non destructive: any existing key is restored and every job it creates is deleted.

`npm run e2e` walks the definition of done against real model calls. The last full run passed all fifteen checks. Selected evidence from that run:

- Re-compiling supersedes rather than duplicating: 48 active against 52 superseded, all 52 carrying a predecessor id, versions reaching 3
- Maximum confidence among unsourced records: 0.40, which is the cap rather than a model's claim
- Coverage gaps: Reddit at rank 4, LinkedIn at rank 5

- A draft with four evidence items and a critic score of 0.95
- A recorded edit distance of 0.0580
- The planner gating an unobserved channel with the reason quoted above
- Editing a cited record marking one artifact stale and creating one successor
- Model calls logged with tokens and cost

`npm run eval` runs a golden set of twenty fixed cases. It departs from the brief it was written against, and says so in the file:

```
/**
 * A deliberate departure from the spec's wording, flagged rather than done
 * silently: the spec asks for "20 fixed prompts with reference outputs", but
 * exact-match reference outputs are meaningless for open-ended drafting.
 */
```

Exact match references would measure temperature, not quality. Each case instead pairs a fixed input with assertions over the output: JSON validity, every record addressable, confidence in range, no uncited record claiming 0.5 or above, no fabricated metrics, no invented thread URLs, and one negative case where a draft stuffed with "trusted by 50,000 developers" and "10x faster" must score below the critic threshold.

Bugs these checks actually caught

Verification is only worth describing if it found something.

Query normalisation. Two spellings of one search query produced two different cache keys and called the provider twice, which is precisely the duplicate execution defect from section 2. The cause was ordering: terminal punctuation was stripped before trimming, so a trailing space kept the anchor from ever matching. It survived until a search API key made the integration check runnable. The fix is three characters of reordering, and the invariant is now a permanent check.

Stage truncation. A compile stage emitted enough records to hit the output token cap, leaving incomplete JSON. The retry then made it worse: told only that the JSON was invalid, the model returned the same too long answer. The fix distinguishes the two cases and tells the retry it has a length problem.

Fixtures leaking into real state. The pair capture script inserted two drafts directly, bypassing the runner that enforces the evidence invariant, and never removed them. The interface flagged them correctly, with "No evidence. This should be impossible", and that warning reached a screenshot in a draft of this document. The script now attaches evidence like any real draft and deletes its fixtures once the pair is captured. The same review found the observation fixture accumulating one row per run, under a caption reading "a single observation".

A runaway recursive query. Introduced while building the derivation graph for this document. `UNION` deduplicates the whole row, so a node reachable at several depths produced one row per depth and diamond shaped paths multiplied out. It hung a real capture run on a 199 edge graph. Carrying the visited path and excluding nodes already on it fixed it, and the same query now returns in under a second.

6. Limits and what comes next

Not implemented

Google Search Console. An optional intake path in the brief. No OAuth flow, no client, no sync job. The `observation_source` enum contains `gsc` and nothing writes it.

GitHub pull requests for blog content. Markdown export exists. Repository integration does not.

The three pane layout. The brief asks for memory left, work centre, agents and jobs right. The interface is one pane per route with a top nav. The information is all present; the layout is not what was asked for.

Per locale agent runs. Voice rules compile per locale and Japanese rules exist in the memory, but the planner hard codes English on every job, so those rules are compiled and never used.

The `classify` task and `settings.model_config`. Both exist because the brief names them. Neither is read by any code path.

Shortcuts, and what they cost

There are no transactions. The Neon HTTP driver runs every statement in its own implicit transaction, so writing a new record version and superseding the old one are two separate operations with a window between them. A crash in that window leaves two active versions. Compiles are serialised by the queue so it is not reachable in normal use, but it is real.

One active record per key is enforced in application code, not the database. The partial unique index that would enforce it cannot be added while the insert happens before the supersede. Fixing it properly means the transaction work above.

Derivation edges over approximate. Every record a stage emits gets an edge to every record in that stage's dependency slice, because the stage saw them together and cannot report which it actually used. This marks slightly more stale than strictly necessary, which is the safer direction, but the count is an upper bound.

A re-compile supersedes records without marking derived artifacts stale. Only human edits propagate. This was found while capturing screenshots for this document: after three re-compiles, no existing draft cited a record that was still active. It is arguably the more common path in production and should propagate too.

The critic's original draft is discarded. When the critic revises a low scoring draft, the pre-revision text is not stored, so the critic's own effect cannot be measured. Only the human's, through edit distance.

No pagination anywhere, in the interface or the API. Lists are capped and a workspace exceeding those caps silently sees a truncated view.

No authentication at all. Every control described in this document assumes a trusted operator. This must not be deployed publicly as it stands.

No unit tests, no UI tests, no CI, nothing hermetic. The three scripts all require a live database, and two require a live model provider. The cost was concrete and is described in section 5.

What I would build next, in order

Move to the WebSocket driver and get transactions. This unblocks the active record unique index, makes editing atomic, and lets the claim query return to its conventional shape. Most of the correctness gaps

above disappear with it, so it comes first.

Propagate staleness on re-compile, not only on human edit. The machinery already exists. The compiler simply does not call it.

Make the compiler resumable. Stages already commit independently. Recording which stages completed against which source version would turn a ten minute all or nothing job into restartable work, which fixes the serverless timeout problem properly rather than working around it.

Persist the compile summary. Per stage counts exist only as text in a job log. A table would make memory quality a trend rather than an anecdote, and the unsourced ratio over time is the best single indicator of whether the crawl and the prompts are improving.

Unit tests for the pure functions. Edit distance, duration parsing, slug generation, URL canonicalisation, query normalisation, the envelope normaliser. All pure, all edge case heavy, all untested, and demonstrably where the bugs have been.

Then authentication, then Search Console, then the two agents the system itself identified. Reddit and LinkedIn are the channels the compiler ranked and the registry cannot serve. Writing them is roughly a day each given the agent contract, and it would demonstrate the extension path on gaps the system found rather than ones chosen in advance.